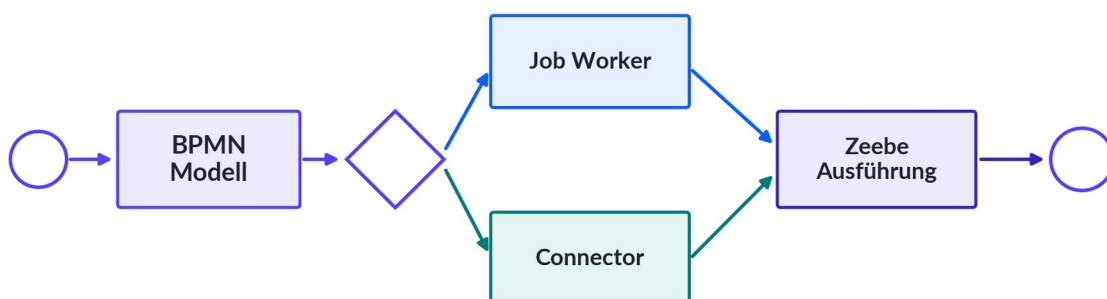


Camunda 7 vs. Camunda 8

BPMN modellieren für Camunda 8

Für erfahrene Camunda-7-Modelliererinnen und -Modellierer



C8 im Mittelpunkt

Vergleichsbasis Camunda 7.24	Zielplattform Camunda 8.9	Stand 24. Juni 2026
---------------------------------	------------------------------	------------------------

Über dieses Handbuch

Dieses Handbuch setzt praktische Erfahrung mit Camunda 7 voraus. Es erklärt nicht BPMN von Grund auf, sondern übersetzt vertraute Camunda-7-Denkmuster in ausführbare Camunda-8-Modelle. Camunda 8 ist der Hauptgegenstand; Camunda 7 dient als Vergleichsfolie.

VERSIONSBEZUG: Alle Verfügbarkeitsaussagen beziehen sich auf Camunda 8.9 und Camunda 7.24. Bei älteren Camunda-8-Ständen können insbesondere Signal-, Eskalations-, Link-, Conditional- und Kompensationsereignisse anders oder noch nicht unterstützt sein. Alte Vergleichslisten sind deshalb häufig überholt. *Quellen: C8-01, C7-01*

Lernziele

- Den Unterschied zwischen modellierbar, ausführbar und nur dokumentarisch nutzbar sicher erkennen.
- Camunda-7-Konfigurationen wie JavaDelegate, JUEL, asyncBefore und eingebettete Formulare in C8-Ausführungsverträge übersetzen.
- Nicht verfügbare Bausteine anhand der fachlichen Absicht ersetzen, ohne die Semantik zu verlieren.
- Nachrichten, Fehler, Retries, Incidents, Variablen und Compensation C8-gerecht modellieren.
- Ein C7-Modell systematisch analysieren, umbauen, testen und für Betrieb und Weiterentwicklung absichern.

Didaktische Nutzung

Lesemodus	Vorgehen
Schnelle Orientierung	Kapitel 1 bis 3 lesen; anschließend die Checklisten in Kapitel 10 verwenden.
Modellierungsworkshop	Je Bausteinfamilie Kapitel 4 bis 8 bearbeiten und das Fallbeispiel in Kapitel 9 parallel nachmodellieren.
Migration Review	Mit der Kreuzmatrix in Kapitel 3 und dem Arbeitsblatt in Kapitel 10 jedes vorhandene C7-Modell klassifizieren.
Nachschlagewerk	Anhang A bis D für Elementstatus, Property-Mapping, FEEL-Muster und Begriffe nutzen.

Inhalt

Kapitel	Thema	Seite
1	Das neue mentale Modell	4
2	Der BPMN Designer in Camunda 8	6
3	Bausteinvergleich und Kreuzmatrizen	8
4	Tasks in Camunda 8	11
5	Daten, Bedingungen, Gateways und Schleifen	14
6	Ereignisse und Abweichungen	16
7	Subprozesse, Wiederverwendung und flexible Arbeit	20
8	Was fehlt oder anders funktioniert	22
9	Durchgängiges Fallbeispiel	25
10	Checklisten, Review und Übungen	28
A	Vollständige Schnellreferenz	32
B	Property-Mapping C7 nach C8	33
C	FEEL-Kurzübersicht	33
D	Glossar	34
E	Quellen und Versionshinweise	34

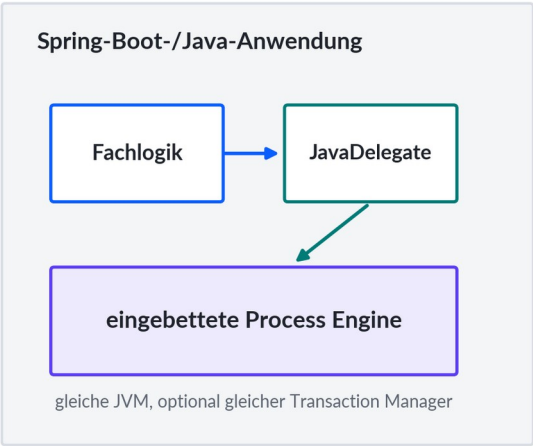
1 Das neue mentale Modell

KERNAUSSAGE: BPMN bleibt die gemeinsame Sprache. Der Ausführungsvertrag ändert sich: C8 orchestriert verteilte Arbeit über Jobs, Nachrichten und APIs. Ein vertrautes Symbol kann deshalb gleich aussehen und dennoch anders konfiguriert oder betrieben werden. *Quellen: MIG-01*

1.1 Vom eingebetteten Motor zum entfernten Orchestrator *Quellen: MIG-01*

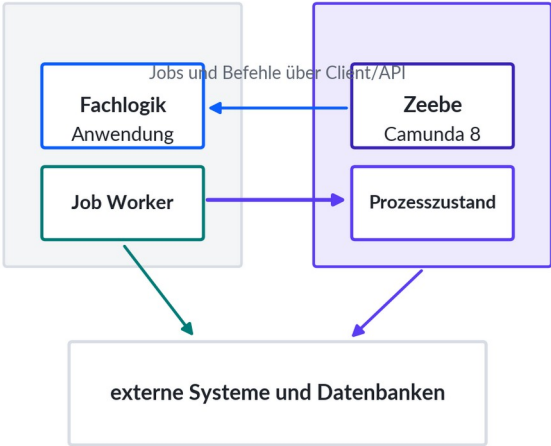
In Camunda 7 konnte die Process Engine als Bibliothek in derselben JVM wie die Fachanwendung laufen. Engine, Delegates und Anwendungsdaten konnten denselben Thread, dieselbe Datenquelle und denselben Transaction Manager teilen. Camunda 8 unterstützt keinen eingebetteten Engine-Modus. Zeebe ist eine entfernte Ressource; Anwendungen kommunizieren über Clients und APIs.

Camunda 7: Engine kann Teil der Anwendung sein



Eine technische ACID-Transaktion kann Engine und Anwendung umfassen.

Camunda 8: Engine ist eine entfernte Ressource



Keine gemeinsame ACID-Transaktion mit dem Workflow-Zustand. Konsistenz entsteht durch Idempotenz, Retries und Kompensation.

Abbildung 1: Architekturwechsel und direkte Modellierungsfolgen

1.2 Die fünf wichtigsten Folgen für das Modell

C7-Denkmuster	C8-Folge für die Modellierung
Delegate läuft im Engine-Thread	Service Task erzeugt einen Job. Worker-Ausführung, Timeout, Retry und Idempotenz gehören zum Lösungsdesign.
Engine und Fachlogik teilen eine ACID-Transaktion	Zwischen Workflow-Zustand und externem System gibt es keine gemeinsame ACID-Transaktion. Fachliche Konsistenz wird über Zustände, Retries und Kompensation erreicht.
Expression kann Spring-Beans aufrufen	FEEL greift auf Prozessdaten zu. Externe Berechnung wird als Worker, Connector oder Execution Listener sichtbar gemacht.
Java-Objekte und Spin im Variablenkontext	Variablen sind Primärtypen oder JSON. Datenverträge und Mappings werden expliziter.
Technische Hooks und Plugins im Engine-Prozess	Hooks werden als Job-basierte Listener, Worker, Interceptor oder Exporter umgesetzt. Engine-Plugins und UI-Plugins sind nicht übertragbar.

1.3 Das C8-Ausführungsdreieck Quellen: C8-02

Der Ausführungsvertrag eines Service Tasks in Camunda 8



Abbildung 2: Lebenszyklus eines Service-Task-Jobs

PRAXISREGEL: Jeder automatisierte Schritt braucht eine klare Antwort auf drei Fragen: Welcher Job-Typ wird erzeugt? Welche Daten werden hinein- und herausgemappt? Was bedeutet fachlicher Fehler gegenüber technischem Fehlschlag?

1.4 Übersetzungswörterbuch

Camunda 7	Camunda 8	Modellierungsbedeutung
JavaDelegate / delegateExpression	Job Worker	Externe Arbeit mit explizitem Job-Typ.
External Task Topic	Job Type	Das Routing-Prinzip wird zum Standard für Service Tasks.
JUEL	FEEL	Bedingungen und Mappings greifen nur auf Prozessdaten zu.
Failed Job	Failed Job, Retries, Incident	Technischer Fehler bleibt außerhalb des fachlichen BPMN-Fehlerpfads.
Task Listener	User Task Listener	Lifecycle der Camunda User Task; Job-basierte Ausführung.
Execution Listener	Execution Listener	Start/End-Hook als blockierender Job, nicht als In-Process-Klasse.
Embedded Form	Camunda Form / externe Form	Ressourcenbindung und Deployment werden bewusst konfiguriert.
Cockpit	Operate	Betriebs- und Incident-Sicht.
Tasklist	Tasklist	Produktname bleibt, Daten- und Autorisierungsmodell unterscheiden sich.

2 Der BPMN Designer in Camunda 8

2.1 Modeler statt bloßem Zeichenprogramm

Der Camunda Modeler besteht weiterhin aus Palette, Canvas, Kontextmenü und Properties Panel. Für C7-Erfahrene ist die Bedienlogik vertraut. Entscheidend ist jedoch die Zielplattform: Erst die gewählte Execution Platform und Version bestimmen, welche Extensions, Validierungen und Laufzeitsemantiken gelten.

Orientierung im Camunda Modeler

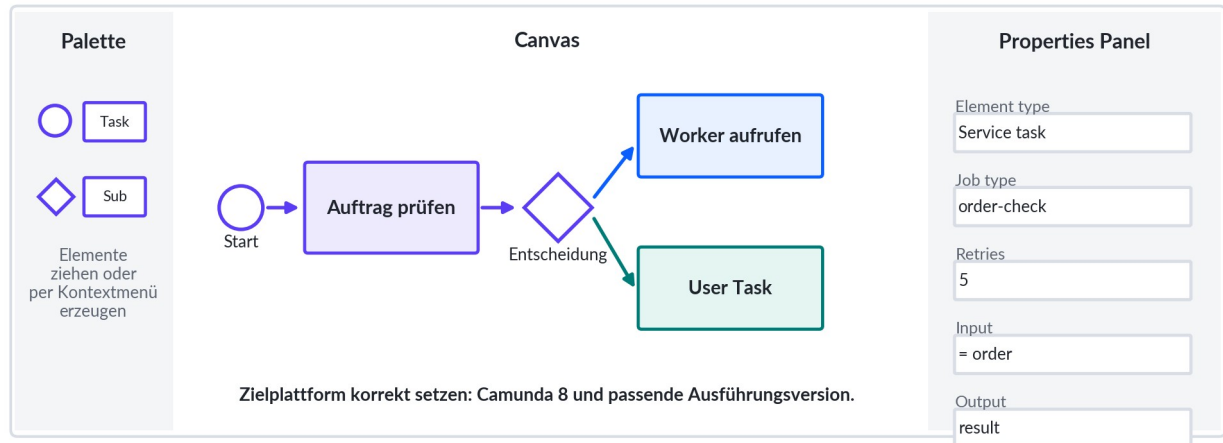


Abbildung 3: Schematische Orientierung im Modeler

2.2 Desktop Modeler und Web Modeler

Aspekt	Desktop Modeler	Web Modeler
Kernmodellierung	Lokale BPMN-, DMN- und Form-Dateien; vertraute Desktop-Arbeitsweise.	Browserbasierte Modellierung mit vergleichbarer Kernbedienung.
Zusammenarbeit	Dateibasiert, typischerweise über Git oder Dokumentenablage.	Kommentare, Versionen, Reviews und gemeinsames Arbeiten im Workspace.
Deployment	Direkt oder über CI/CD, abhängig von Umgebung und Einstellungen.	Integrierte Deployments und Process Applications, abhängig vom Betriebsmodell.
Element Templates	Lokal oder projektspezifisch verwaltet.	Workspace- und projektbezogen nutzbar.
Geeignet für	Lokale Entwicklung, Git-zentrierte Teams, Offline-Arbeit.	Fachlich-technische Kollaboration und zentral organisierte Artefakte.

2.3 Die drei Ebenen eines C8-Elements

Ebene	Leitfrage	Beispiel Service Task
BPMN-Semantik	Welche fachliche Arbeit oder welcher Wartezustand wird dargestellt?	„Zahlung autorisieren“ ist eine automatisierte Aktivität.
Camunda-Extension	Wie wird das Element ausführbar konfiguriert?	Job Type payment-authorize, Retries 5, Input/Output Mappings.
Implementierung	Welcher technische Baustein erfüllt den Vertrag?	Java-/Node-/C#-Worker oder Connector.

ACHTUNG: Ein Element Template oder Connector ersetzt nicht die BPMN-Semantik. Es vorkonfiguriert ein Element, meist einen Service Task. Bei Reviews müssen Symbol, Extension und Implementierung getrennt geprüft werden.

2.4 Modeler-Workflow für C7-Umsteiger

1. Execution Platform auf Camunda 8 und die vorgesehene Version setzen.
2. Zuerst die fachliche BPMN-Semantik wählen, danach Implementierung oder Template.
3. Properties Panel vollständig konfigurieren: Job Type, Retries, Bindings, Mappings, Korrelation und Formreferenz.
4. Warnungen nicht als kosmetisch behandeln. Nicht ausführbare Elemente können im Diagramm sichtbar bleiben und erst beim Deployment scheitern.
5. Diagramm und Ressourcen als Einheit versionieren: BPMN, DMN, Form, Element Templates und Worker-Vertrag.

2.5 Modellierbar ist nicht gleich ausführbar Quellen: C8-01

Status im Handbuch	Bedeutung	Konsequenz
Ausführbar	Camunda 8.9 verarbeitet das Element mit dokumentierter Laufzeitsemantik.	Kann nach vollständiger Konfiguration deployed werden.
Nur Modellierung	Das Symbol dokumentiert, erzeugt aber keinen Laufzeitzustand oder keine Datenwirkung.	Nicht als technische Umsetzung missverstehen.
Nicht ausführbar	Das Symbol kann teilweise gezeichnet oder importiert werden, wird aber nicht von Zeebe ausgeführt.	Vor Deployment durch ein C8-Muster ersetzen.
Nicht vorgesehen	Diese Kombination existiert semantisch in BPMN nicht, etwa ein werfender Timer.	Eine andere Ereignisart oder Platzierung wählen.

2.6 Properties-Panel-Check vor dem Deployment

Prüfebene	Kontrollfrage
Semantik	Ist das BPMN-Element der fachlichen Absicht entsprechend gewählt?
Zielformat	Sind Camunda 8 und die tatsächlich betriebene Ausführungsversion gesetzt?
Ausführung	Sind Job Type, Binding, Formreferenz oder Message-Konfiguration vollständig?
Daten	Sind Input, Output, lokale Variablen und Scope-Wirkung ausdrücklich geprüft?
Abweichung	Sind Business Error, technischer Fail, Retries und Incident-Betrieb getrennt?

3 Bausteinvergleich und Kreuzmatrizen

LESESCHLÜSSEL: Die Tabellen vergleichen Engine-Ausführung, nicht bloße Darstellbarkeit im Modeler. „Anders“ bedeutet: Das BPMN-Symbol bleibt, aber die Extension oder operative Semantik ändert sich.

3.1 Tasks Quellen: C8-01, C7-01

Element	C7.24	C8.9	Delta	C8-Modellierung
Service Task	Ausführbar	Ausführbar	Stark anders	Job Type plus Worker oder Connector; JavaDelegate-Konfiguration entfällt.
User Task	Ausführbar	Ausführbar	Anders	Camunda User Task verwenden; Job-Worker-Variante ist in 8.9 deprecated.
Receive Task	Ausführbar	Ausführbar	Anders	Message Name und Correlation Key als FEEL konfigurieren.
Send Task	Ausführbar	Ausführbar	Anders	Verhält sich technisch wie Service Task; Versand durch Worker/Connector.
Business Rule Task	Ausführbar	Ausführbar	Anders	DMN mit Binding latest/deployment/versionTag oder Worker.
Script Task	Ausführbar	Ausführbar	Stark anders	Nativ Inline-FEEL; Groovy, JavaScript oder Python über Worker.
Manual Task	Ausführbar als Pass-through	Ausführbar als Pass-through	Gleich	Keine Engine-Unterstützung für die manuelle Arbeit; kein Warten.
Undefined Task	Nicht als ausführbar markiert	Ausführbar als Pass-through	C8 erweitert	Nur für Dokumentation/Entwurf; nicht als Wartezustand.
Instantiating Receive Task	Nicht ausführbar	Nicht ausführbar	Kein C8-Verlust	Message Start Event verwenden.

3.2 Gateways, Subprozesse und Marker Quellen: C8-01, C7-01

Element	C7.24	C8.9	C8-Hinweis
Exclusive Gateway	Ausführbar	Ausführbar	Bedingungen von JUEL nach FEEL umstellen.
Parallel Gateway	Ausführbar	Ausführbar	Variablenkonflikte in parallelen Pfaden explizit vermeiden.

Element	C7.24	C8.9	C8-Hinweis
Inclusive Gateway	Ausführbar	Ausführbar	FEEL-Bedingungen; sauberer Default Flow.
Event-based Gateway	Ausführbar	Ausführbar	In C8 ausgehend nur zu Message-, Timer- oder Signal-Catch-Events.
Complex Gateway	Nicht ausführbar	Nicht ausführbar	Zerlegen oder Entscheidung/Aggregation vorab berechnen.
Embedded Subprocess	Ausführbar	Ausführbar	Für Scope, Variablenkapselung und Boundary Events.
Call Activity	Ausführbar	Ausführbar	Binding und Variable Propagation bewusst setzen.
Event Subprocess	Ausführbar	Ausführbar	C8 unterstützt Timer, Message, Error, Signal und Escalation als Start.
Transaction Subprocess	Ausführbar	Nicht ausführbar	Saga/Embedded Subprocess plus Kompensation.
Ad-hoc Subprocess	Nicht ausführbar	Ausführbar	C8-Gewinn für flexible, dynamisch aktivierte Arbeit.
Parallel/Sequential Multi-Instance	Ausführbar	Ausführbar	Collection, Element, Output und Completion Condition per FEEL.
Standard Loop Marker	Nicht ausführbar	Nicht ausführbar	Rücksprung oder Multi-Instance.
Compensation Marker	Ausführbar	Ausführbar	Semantik bei Reihenfolge und Multi-Instance unterscheidet sich.
Ad-hoc Marker	Nicht ausführbar	Ausführbar	Nur im Ad-hoc Subprocess.

3.3 Ereignisfamilien im Überblick Quellen: C8-01, C7-01

Familie	C7.24	C8.9	Zentrale Aussage
None	Start, Throw, End	Start, Throw, End	Gleich
Message	Start, Event Subprocess, Catch, Boundary, Throw, End	Start, Event Subprocess, Catch, Boundary, Throw, End	Korrelation und TTL C8-spezifisch konfigurieren
Timer	Start, Event Subprocess, Catch, Boundary	Start, Event Subprocess, Catch, Boundary	Ausdrücke und Wiederholung mit C8-Semantik prüfen
Conditional	Start, Event Subprocess, Catch, Boundary	Start, Event Subprocess, Catch, Boundary	Bedingung als FEEL; reagiert auf Prozessdaten
Signal	Start, Event Subprocess, Catch, Boundary, Throw, End	Start, Event Subprocess, Catch, Boundary, Throw, End	Broadcast-Semantik
Error	Event Subprocess, Boundary, End	Event Subprocess, Boundary, End	Business Error von Job Failure trennen
Escalation	Event Subprocess, Boundary, Throw, End	Event Subprocess, Boundary, Throw, End	Nicht-kritische Meldung an höheren Scope
Terminate	End	End	Beendet alle Instanzen im gleichen Flow Scope
Link	Catch, Throw	Catch, Throw	Nur innerhalb desselben Scopes
Compensation	Event Subprocess, Boundary, Throw, End	Boundary, Throw, End	Compensation Event Subprocess fehlt in C8

Familie	C7.24	C8.9	Zentrale Aussage
Cancel	Boundary, End im Transaction Subprocess	Nicht ausführbar	Saga-/Kompensationsmuster
Multiple / Parallel Multiple	Nicht ausführbar	Nicht ausführbar	Explizite Gateways und Events verwenden

3.4 Was C8 tatsächlich verliert, gewinnt oder nur anders macht

Kategorie	Elemente oder Fähigkeit	Einordnung
C8 nicht ausführbar, C7 ausführbar	Transaction Subprocess, Cancel Events, Compensation Event Subprocess	Echte Umbaumuster erforderlich.
In beiden nicht ausführbar	Complex Gateway, Standard Loop, Instantiating Receive Task, Multiple Events	Kein C8-Rückschritt; bestehende Modelle waren bereits nicht enginefähig.
C8 hinzugekommen	Ad-hoc Subprocess, Ad-hoc Marker, Undefined Task als Pass-through	Neue Modellierungsoptionen, insbesondere für flexible Arbeit.
Symbol bleibt, Vertrag ändert sich	Service, Script, Send, Business Rule, User Task, Bedingungen, Listener	Größter Schulungsbedarf: gleiche Notation, andere Properties und Betriebssemantik.
Nur Diagramm in beiden	Data Object, Data Store, Text Annotation, Group	Keine Prozessvariablen oder Persistenzwirkung.

3.5 Migrationsrisiko priorisieren

Risikoklasse	Typische Fundstelle	Behandlung
Sehr hoch	Transaction/Cancel, Engine-Plugins, implizite ACID-Erwartung	Fachliche Konsistenz neu entwerfen; Architektur- und Betriebsentscheidung einbeziehen.
Hoch	JavaDelegate, Bean-Ausdrücke, Kompensationsreihenfolge, eingebettete Forms	Ausführungsvertrag und Ressourcenbindung explizit neu modellieren.
Mittel	JUEL-Bedingungen, Variable Propagation, Message-Korrelation, Listener	Syntax, Scope und Lifecycle systematisch testen.
Niedrig	Reine Dokumentation, Labels, Pools/Lanes ohne technische Erweiterung	Semantik prüfen und kontrolliert übernehmen.

4 Tasks in Camunda 8

Welcher Task-Typ passt in Camunda 8?

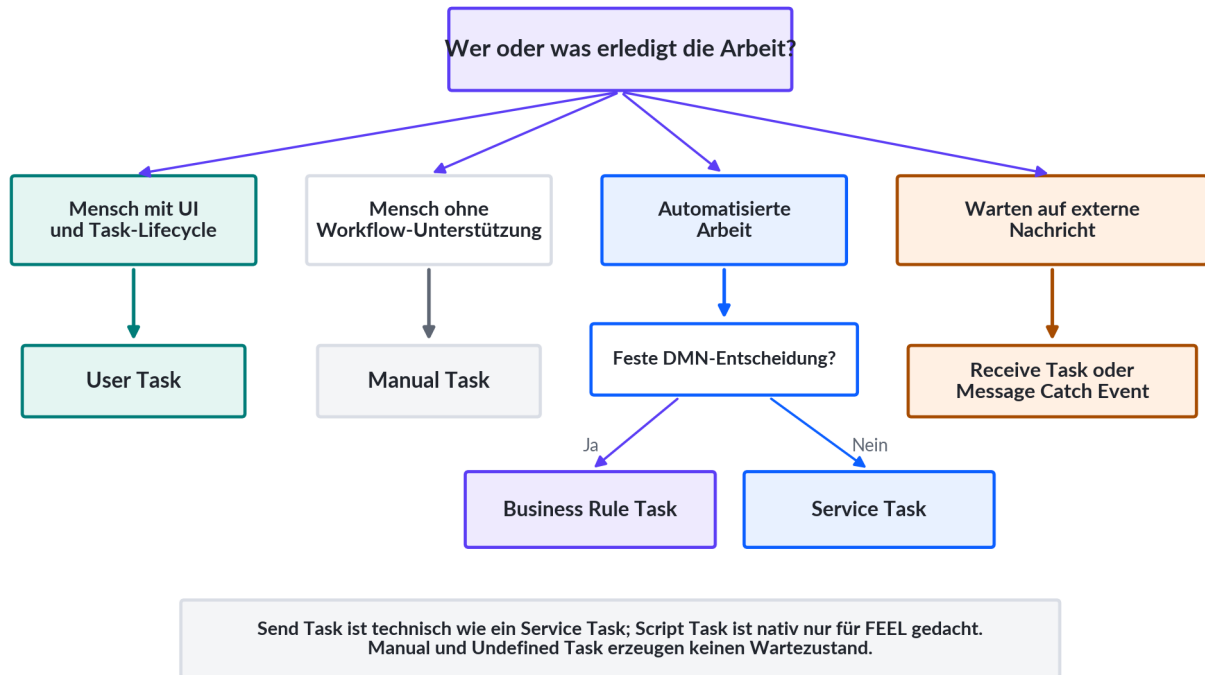


Abbildung 4: Entscheidungshilfe für Task-Typen

4.1 Service Task: der Standard für automatisierte Arbeit Quellen: C8-02

Beim Eintritt erzeugt Zeebe einen Job und wartet auf dessen Abschluss. Der Task Definition Type routet den Job zum Worker. Der Typ kann statisch oder als FEEL-Ausdruck angegeben werden. Die Task Definition enthält außerdem die initiale Retry-Anzahl; Standardwert ist drei. Task Headers transportieren statische Metadaten, Input/Output Mappings formen den Datenvertrag.

Property	C8-Bedeutung	Reviewfrage
Type	Job-Typ für Worker-Abonnement.	Ist der Name stabil, fachlich verständlich und nicht an eine konkrete Klasse gekoppelt?
Retries	Initiale Anzahl technischer Wiederholungen.	Passt die Zahl zur Fehlerart und zur externen Operation?
Task Headers	Statische Konfiguration für den Worker.	Gehört der Wert ins Modell oder in Betriebs-/Secret-Konfiguration?
Input Mapping	Lokaler, minimaler Worker-Input.	Werden unnötige oder sensible Variablen ausgeschlossen?
Output Mapping	Kontrollierte Rückgabe in den Parent Scope.	Kann ein paralleler Pfad Werte überschreiben?

Beispielkonfiguration im Properties Panel

```

Type: payment-authorize
Retries: 5
Input: orderId == order.id
      amount == order.total
Output: payment == result
  
```

MODELLIERUNGSREGEL: Job-Typen beschreiben eine Fähigkeit, nicht eine Implementierung. payment-authorize ist stabiler als PaymentDelegateV2.

4.2 C7-Implementierungen auf C8 abbilden Quellen: MIG-01

C7 Service-Task-Konfiguration	C8-Ziel	Hinweis
camunda:class	Job Worker	Klasse wird aus BPMN entfernt; Worker registriert sich auf Job Type.
camunda:delegateExpression	Job Worker	Spring Bean bleibt möglich, aber als Worker außerhalb der Engine.
camunda:expression	FEEL oder Worker	Einfache Datenberechnung in FEEL; Bean-/Serviceaufruf als Worker.
camunda:type="external" plus topic	Job Worker Type	Konzeptionell am ähnlichsten; C8 macht dieses Muster zum Standard.
Camunda Connect HTTP/SOAP	C8 Connector oder Worker	Connector-Infrastruktur ist nicht kompatibel und muss neu konfiguriert werden.
asyncBefore/asyncAfter	Kein direktes Attribut	Service Task ist bereits ein Job-Wartepunkt; zusätzliche Grenze durch expliziten Task/Listener modellieren.
exclusive	Kein direktes Attribut	Parallelität über Scopes, Variable Mappings und idempotente Worker beherrschen.

4.3 Send Task Quellen: C8-07

Ein Send Task hat in Camunda 8 dieselbe technische Ausführung wie ein Service Task: Zeebe erzeugt einen Job, ein Worker oder Connector sendet die Nachricht, und der Prozess wartet auf Complete. Der Unterschied ist semantisch und visuell. Verwenden Sie den Send Task, wenn das fachliche Ergebnis eindeutig das Senden einer Nachricht ist; verwenden Sie den Service Task für allgemeine Systemarbeit.

4.4 Business Rule Task Quellen: C8-05

Der Business Rule Task kann eine in Camunda deployte DMN-Entscheidung auswerten oder als Job für einen Worker konfiguriert werden. Für DMN sind Decision ID, Binding und Result Variable zentral. Das Binding deployment koppelt Prozess und Entscheidung aus demselben Deployment; latest bindet dynamisch an die beim Aktivierungszeitpunkt neueste Version; versionTag bindet an einen fachlichen Tag.

Binding	Wirkung	Typischer Einsatz
deployment	Verwendet die zusammen mit dem Prozess deployte Ressource.	Reproduzierbare Releases und gemeinsame Tests.
latest	Verwendet die beim Aufruf neueste deployte Version.	Bewusst dynamische Regeländerung; stärkere Governance nötig.
versionTag	Verwendet die neueste Version mit einem bestimmten Tag.	Fachliche Release-Linie unabhängig von Versionsnummern.

4.5 Script Task Quellen: C8-06

Camunda 8 führt Inline-Skripte nativ als FEEL-Ausdruck aus. Damit eignet sich der Script Task für kurze, deterministische Transformationen auf Prozessdaten. Groovy, JavaScript, Python oder andere Laufzeiten werden nicht in Zeebe eingebettet; dafür ist ein Worker erforderlich. Große Transformationen, externe Zugriffe und schwer testbare Logik gehören ebenfalls in einen Worker.

Geeignet für Inline-FEEL	Nicht geeignet
Feld ableiten, Liste filtern, Kontext erzeugen, einfache Validierung.	Netzwerkzugriff, Dateizugriff, Secrets, lange Berechnung, Bibliothekslogik.
Deterministische Berechnung aus vorhandenen Prozessvariablen.	Seiteneffekte oder eine Operation, die wiederholt werden kann.

4.6 User Task Quellen: C8-03

Die Camunda User Task ist seit 8.6 der empfohlene und standardmäßige Implementierungstyp. Sie erzeugt eine User-Task-Instanz in Zeebe und unterstützt Assignment, Scheduling, Updates, Variable Mappings, Formulare und User Task Listeners. Die ältere Job-Worker-Implementierung wird in 8.9 als deprecated geführt.

Thema	C8-Konfiguration	Typischer C7-Stolperstein
Assignment	assignee, candidateUsers, candidateGroups; statisch oder FEEL.	JUEL- oder Bean-Ausdrücke müssen auf Daten/FEEL umgestellt werden.
Identitäten	Benutzer- und Gruppen-IDs sind case-sensitive.	Gruppennamen und IDs nicht ungeprüft übernehmen.
Form	Camunda Form oder externe Referenz; Binding konfigurieren.	Embedded HTML/JS Forms sind kein 1:1-Artefakt.
Terminierung	dueDate und followUpDate als Datum-/Zeit-Ausdruck.	Datumsformat und Zeitzone testen.
Priorität	Numerischer Wert 0 bis 100.	C7-Prioritätslogik nicht blind übertragen.
Lifecycle Hooks	User Task Listeners für creating, assigning, updating, completing, canceling.	TaskListener-Klasse wird zum Job-basierten Listener.

REVIEW: Eine User Task ist nicht nur ein Rechteck mit Gruppenname. Form-Binding, Assignment-Ausdruck, Sichtbarkeit, Variablen-Mapping, Due Date und Listener bilden gemeinsam den ausführbaren Vertrag.

4.7 Receive Task Quellen: C8-04

Der Receive Task wartet auf eine Nachricht und ist semantisch einem Intermediate Message Catch Event gleichwertig. Der Receive Task ist sinnvoll, wenn der Wartezustand als Arbeitsschritt benannt werden soll oder Boundary Events benötigt werden. Für einen kompakten Ereignisfluss genügt häufig das Catch Event.

4.8 Manual und Undefined Task Quellen: C8-08, C8-09

Beide Task-Typen werden in Camunda 8 als Pass-through behandelt: Der Prozess läuft beim Erreichen sofort weiter. Sie dokumentieren Arbeit außerhalb der Engine, erzeugen aber keine Aufgabe, kein UI und keinen Wartezustand. Ein fachliches Warten auf eine Person muss als User Task oder als explizites Ereignis modelliert werden.

5 Daten, Bedingungen, Gateways und Schleifen

5.1 Von JUEL zu FEEL Quellen: MIG-01

Camunda 7 verwendet für BPMN-Ausdrücke überwiegend JUEL und kann in eingebetteten Szenarien auf Java-Beans zugreifen. Camunda 8 verwendet FEEL; Ausdrücke sehen Prozessvariablen und lokale Variablen, aber keine Spring-Beans. Reine Datenlogik lässt sich oft direkt übersetzen. Serviceaufrufe oder komplexe Bean-Methoden werden zu einem Worker oder Listener.

Absicht	C7/JUEL	C8/FEEL
Betrag größer 1.000	<code>\${amount > 1000}</code>	<code>= amount > 1000</code>
VIP und hoher Betrag	<code>\${customer.vip && amount > 1000}</code>	<code>= customer.vip and amount > 1000</code>
Status in Liste	<code>\${allowed.contains(status)}</code>	<code>= list contains(allowed, status)</code>
Fallback bei null	<code>\${value != null ? value : "n/a"}</code>	<code>= if value = null then "n/a" else value</code>
Kontext erzeugen	Bean oder Script	<code>= { orderId: order.id, amount: order.total }</code>
Service aufrufen	<code>\${riskBean.score(order)}</code>	Service Task oder Execution Listener

5.2 Variablen sind scope-basiert, nicht token-basiert Quellen: C8-10

Prozessvariablen gehören zu Scopes. Parallele Tokens teilen daher sichtbare Variablen. Schreibt ein paralleler Pfad denselben Namen, kann ein anderer Pfad den Wert sehen oder überschreiben. Input Mappings erzeugen lokale Variablen; Output Mappings kontrollieren die Propagation. Worker sollten nur benötigte Variablen abrufen und nur geänderte Werte zurückgeben.

Risiko	Ursache	C8-Gegenmaßnahme
Lost Update	Zwei parallele Jobs schreiben dieselbe Variable.	Lokale Variablen plus getrennte Output-Namen; später bewusst zusammenführen.
Ungewolltes Datenleck	Worker erhält standardmäßig mehr Variablen als nötig.	<code>fetchVariables</code> und Input Mapping begrenzen.
Child überschreibt Parent	Call Activity propagiert alle Child-Variablen.	<code>propagateAllChildVariables</code> deaktivieren oder Output Mapping verwenden.
Multi-Instance-Race	Instanzen schreiben in denselben Parent Scope.	<code>inputElement</code> lokal halten; <code>outputCollection/outputElement</code> nutzen.

5.3 Gateways

Gateway	C8-Verwendung	Häufiger Fehler
Exclusive	Genau ein Pfad; Bedingungen werden in Modellreihenfolge ausgewertet, Default für Rest.	Überlappende Bedingungen ohne dokumentierte Priorität.
Parallel	Alle ausgehenden Pfade; Join wartet auf eingehende Tokens gemäß Modell.	Als „technische Parallelisierung“ ohne Daten- und Fehlerkonzept verwenden.
Inclusive	Alle Pfade mit wahrer Bedingung; Join synchronisiert aktivierte Pfade.	Kein Default und keine Bedingung wahr, dadurch Incident.
Event-based	Erstes eintreffendes Message-, Timer- oder Signal-Catch-Event gewinnt.	Receive Task oder beliebige Aktivität direkt anschließen.

KERNAUSSAGE: Ein Gateway entscheidet oder synchronisiert. Es führt keine Fachlogik aus. Muss zuerst komplex gerechnet werden, gehört die Rechnung in FEEL, DMN, einen Worker oder einen Start-Execution-Listener. Quellen: C8-24

5.4 Multi-Instance Quellen: C8-11

Camunda 8 unterstützt parallele und sequenzielle Multi-Instance-Aktivitäten für alle unterstützten Aktivitäten. `inputCollection` liefert eine Liste, `inputElement` benennt das lokale aktuelle Element. `outputElement` und `outputCollection` sammeln Ergebnisse in der Reihenfolge der Eingabeliste, auch bei paralleler Ausführung. `completionCondition` kann verbleibende Instanzen beenden, sobald die Bedingung erfüllt ist.

Multi-Instance-Konfiguration

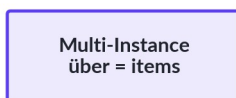
```
Input collection:  = order.items
Input element:    item
Output element:   = result
Output collection: processedItems
Completion:        = numberOfCompletedInstances / numberOfInstances >= 0.8
```

ACHTUNG: Bei einem interrupting Boundary Event werden alle aktiven Instanzen beendet; eine teilweise `outputCollection` wird nicht in den Parent Scope propagiert. Quellen: C8-11

5.5 Schleifen ohne Loop Marker

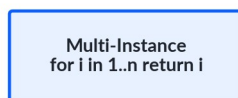
Schleifen ohne Standard Loop Marker

Endliche Menge

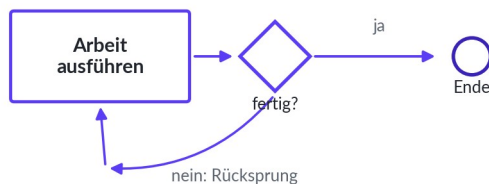


parallel oder sequenziell

Genau n Wiederholungen



Bis Bedingung erfüllt



Schleifen müssen einen stabilen Wartezustand enthalten, wenn externe Arbeit beteiligt ist.
Eine enge, rein interne Rechenschleife gehört eher in einen Worker.
Link Events können lange Rücksprunglinien optisch ersetzen, ändern aber die Semantik nicht.

Abbildung 5: Drei C8-Muster für Wiederholung

Der Standard Loop Marker ist weder in Camunda 7.24 noch in Camunda 8.9 enginefähig. Für eine endliche Liste ist Multi-Instance das klarste Muster. Für „wiederholen bis“ wird ein expliziter Rücksprung über XOR und Sequence Flow modelliert; Link Events können lange Linien ersetzen. Eine enge Berechnungsschleife ohne fachlichen Wartezustand gehört in den Worker.

6 Ereignisse und Abweichungen

Welches Ereignis beschreibt die fachliche Absicht?

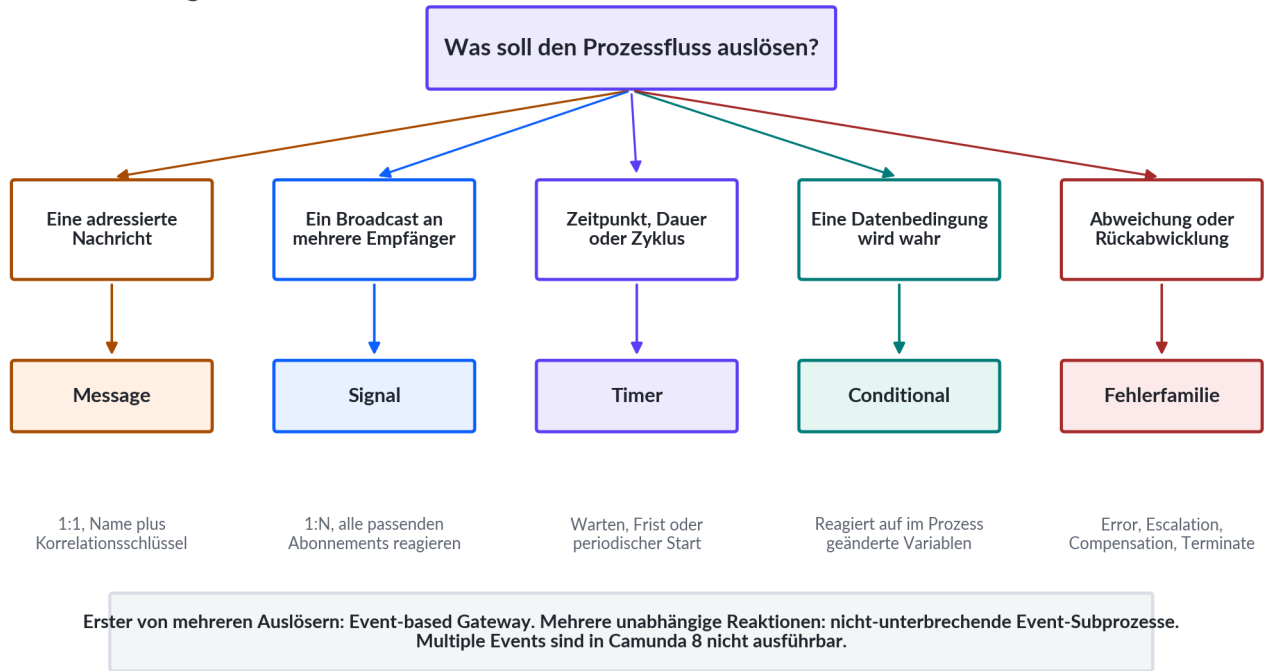


Abbildung 6: Ereigniswahl nach fachlicher Absicht

6.1 Message versus Signal Quellen: C8-12, C8-13

Kriterium	Message	Signal
Empfänger	Genau eine passende Prozessinstanz oder ein Startabonnement.	Alle passenden Abonnements; Broadcast.
Adressierung	Message Name plus Correlation Key.	Signal Name.
Pufferung	Publizierte Message kann mit TTL gepuffert werden.	Broadcast wirkt auf aktuell passende Abonnements.
Typischer Einsatz	Zahlung bestätigt, Dokument eingegangen, Auftrag storniert.	Systemweite Richtlinie geändert, globaler Stopp, mehrere Prozesse informieren.
Hauptrisiko	Falscher/instabiler Korrelationsschlüssel oder Race vor Subscription.	Unbeabsichtigte Massenreaktion.

Nachrichtenkorrelation in Camunda 8

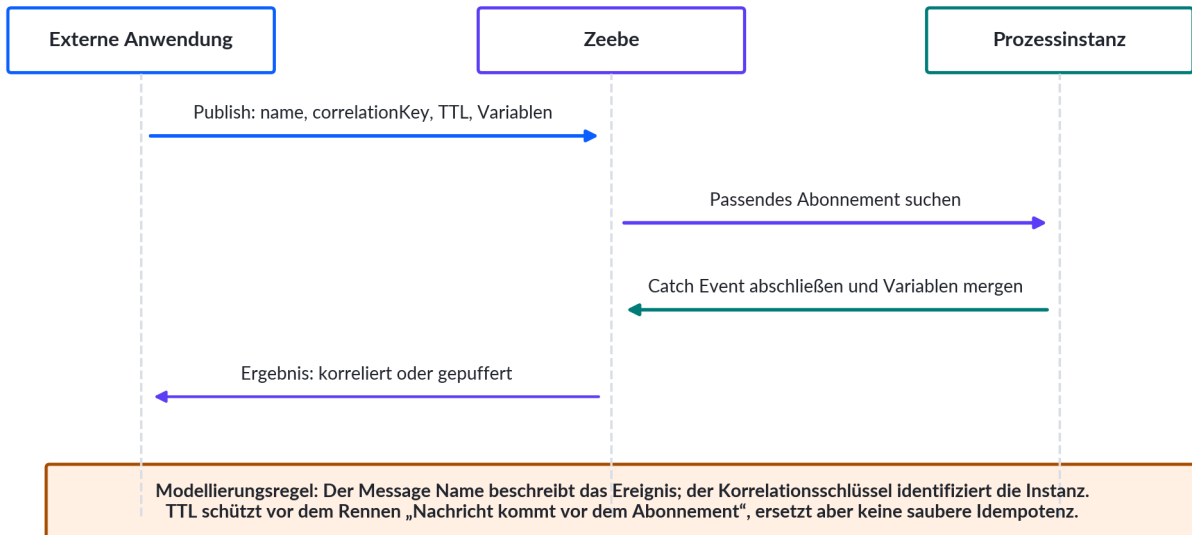


Abbildung 7: Nachrichtenpublikation und Korrelation

6.2 Start Events und Korrelation Quellen: C8-12

Message Start Events erzeugen beim Deployment Abonnements. Eine publizierte Nachricht mit passendem Namen startet eine Instanz. Ein mitgelieferter Correlation Key kann sicherstellen, dass pro Schlüssel nur eine aktive Instanz existiert; gepufferte Nachrichten mit TTL können nach Abschluss der aktiven Instanz eine neue Instanz starten. Im BPMN-Start-Event wird kein Correlation-Key-Ausdruck konfiguriert; der Sender liefert den Wert.

6.3 Timer und Conditional Events Quellen: C8-14, C8-15

Ereignis	Verwenden wenn	Nicht verwenden wenn
Timer	Ein Zeitpunkt, eine Dauer, ein Zyklus oder eine Frist ist die Ursache.	Ein externes System irgendwann einen Status ändert, ohne dass der Prozess Variablen aktualisiert.
Conditional	Eine in Prozessdaten ausdrückbare Bedingung durch Variablenänderung wahr wird.	Die Bedingung nur durch Polling eines externen Systems erkannt werden kann; dafür Worker plus Timer.

6.4 BPMN Error, Job Failure und Incident Quellen: C8-16, C8-02

Abweichungen richtig klassifizieren

Fachlich erwartete Ausnahme

Worker wirft BPMN Error; Boundary Event oder Error Event Subprocess modelliert die Reaktion.

Nicht-kritische fachliche Meldung

Escalation kommuniziert nach oben; der werfende Pfad läuft weiter.

Technischer, vorübergehender Fehler

Worker meldet Fail; Retries werden reduziert. Nach Ausschöpfung entsteht ein Incident.

Irreparabler Modell-/Datenfehler

Incident beheben, Daten oder Modell korrigieren und Ausführung fortsetzen.

Nicht jeder Fehler ist ein BPMN Error.

Abbildung 8: Klassifikation von Abweichungen

Situation	Worker-Antwort	BPMN-Reaktion
Kreditkarte fachlich abgelehnt	Throw BPMN Error mit Error Code.	Boundary Error oder Error Event Subprocess.
Payment API temporär nicht erreichbar	Fail Job; Retries reduzieren, Backoff beachten.	Kein Error Event; nach Retries Incident.
FEEL-Ausdruck greift auf fehlende Variable zu	Engine erzeugt Incident.	Daten/Modell korrigieren und fortsetzen.
Nicht-kritische Sonderlage an übergeordneten Scope	Escalation werfen.	Escalation Boundary/Event Subprocess; werfender Pfad läuft weiter.

ACHTUNG: Ein BPMN Error ist eine fachlich modellierte Ausnahme, kein Ersatz für jeden technischen Fehler. Wird jede Netzwerkstörung als BPMN Error behandelt, werden Retries und Incident-Betrieb umgangen.

6.5 Escalation Quellen: C8-17

Escalation kommuniziert eine nicht-kritische Situation an einen höheren Flow Scope. Anders als beim Error wird der werfende Pfad nicht abgebrochen; sowohl der ausgehende Pfad des Throw Events als auch der Pfad des Catch Events können weiterlaufen. Das Muster eignet sich für Benachrichtigung, zusätzliche Prüfung oder parallele Aufsicht, nicht für eine harte Ausnahme.

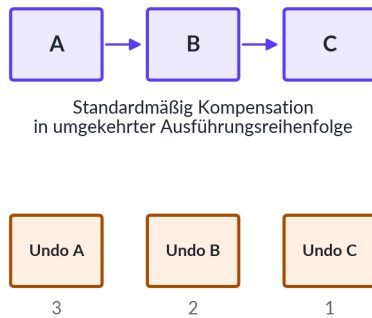
6.6 Terminate und Link Quellen: C8-18, C8-19

Ein Terminate End Event beendet alle aktiven Elementinstanzen im gleichen Flow Scope. Es ist ein scharfes Werkzeug und sollte nur eingesetzt werden, wenn konkurrierende Pfade tatsächlich entfallen müssen. Link Events verbinden zwei Stellen innerhalb desselben Scopes. Sie sind optische Abkürzungen und können Rücksprünge oder lange Sequence Flows lesbarer machen; sie sind kein Nachrichtenmechanismus.

6.7 Kompensation Quellen: C8-20, C7-02

Kompensation: eine wichtige Semantikänderung

Camunda 7



Camunda 8

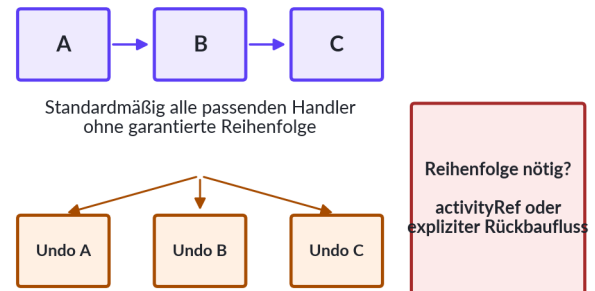


Abbildung 9: Unterschiedliche Standardreihenfolge bei Kompensation

In Camunda 8 wird ein Compensation Boundary Event an eine erfolgreich abschließbare Aktivität gehängt und über eine Association mit dem Compensation Handler verbunden. Ein Compensation Throw oder End Event aktiviert passende Handler im Scope. Standardmäßig werden mehrere Handler gleichzeitig und ohne garantierte Reihenfolge aktiviert. Ist die Reihenfolge fachlich relevant, muss gezielt eine Aktivität per `activityRef` kompensiert oder ein expliziter Rückabwicklungsfluss modelliert werden.

Aspekt	Camunda 7	Camunda 8
Reihenfolge mehrerer Handler	Standardmäßig umgekehrte Ausführungsreihenfolge.	Alle passenden Handler ohne garantierte Reihenfolge.
Multi-Instance-Aktivität	Subscription pro erfolgreicher Instanz; Handler entsprechend mehrfach.	Handler der Multi-Instance-Aktivität wird einmal aufgerufen und muss alle Instanzen rückgängig machen.
Compensation Event Subprocess	Unter Einschränkungen unterstützt.	Nicht ausführbar.
Call Activity	Kompensation propagiert nicht in Child-Instanz.	Stoppt ebenfalls an Call Activity; Handler an Call Activity anbringen.

7 Subprozesse, Wiederverwendung und flexible Arbeit

7.1 Embedded Subprocess

Ein Embedded Subprocess gruppiert Elemente, erzeugt einen Scope und erlaubt Boundary Events sowie lokale Variablen. Er benötigt genau ein None Start Event. Collapsed und Expanded sind Darstellungsvarianten, keine Ausführungsunterschiede. In C8 ist er das zentrale Gefäß für Saga-Schritte, Fehlergrenzen und Datenkapselung.

7.2 Event Subprocess Quellen: C8-22

Ein Event Subprocess wird durch Timer, Message, Error, Signal oder Escalation gestartet. Er kann interrupting oder non-interrupting sein. Weil er innerhalb des Scopes liegt, kann er dessen lokale Variablen lesen und verändern. Das unterscheidet ihn vom Boundary Event, das außerhalb des Scopes liegt. Non-interrupting Event Subprocesses können mehrfach ausgelöst werden; interrupting nur einmal.

Bedarf	Boundary Event	Event Subprocess
Reaktion an genau einer Aktivität	Direkt an Aktivität anheften.	Unnötig.
Reaktion gilt für gesamten Scope	Am Subprozessrand möglich.	Im Scope meist lesbarer, insbesondere mit mehreren Schritten.
Zugriff auf lokale Scope-Variablen	Eingeschränkt, weil außerhalb des Scopes.	Direkter Zugriff innerhalb des Scopes.
Mehrstufige Reaktion	Pfad außerhalb des Elements.	Eigener interner Teilprozess.

7.3 Call Activity und Ressourcenbindung Quellen: C8-21

Eine Call Activity startet eine neue Prozessinstanz am None Start Event des referenzierten Prozesses. Andere Start Events des Child-Prozesses werden für diesen Aufruf ignoriert. processId kann statisch oder als FEEL-Ausdruck bestimmt werden. Binding, Parent-to-Child-Propagation und Child-to-Parent-Propagation müssen als API-Vertrag behandelt werden.

Reviewpunkt	Sichere C8-Praxis
Binding	Für gemeinsam getestete Releases deployment; latest nur bei bewusst dynamischer Kopplung.
Input	propagateAllParentVariables bei Bedarf deaktivieren; lokale Input Mappings als Child-API.
Output	propagateAllChildVariables deaktivieren oder Output Mappings verwenden, besonders parallel.
Boundary Event	Interrupting beendet Call Activity und Child-Instanz; Child-Variablen werden dann nicht propagiert.
Kompensation	Kompensation stoppt an Call Activity; Rückbau über Handler an der Call Activity orchestrieren.

7.4 Ad-hoc Subprocess Quellen: C8-23

Der Ad-hoc Subprocess ist ein C8-Gewinn gegenüber der C7-Engine-Coverage. Seine inneren Aktivitäten benötigen keine Start- oder End Events und können in beliebiger Reihenfolge, mehrfach oder gar nicht ausgeführt werden. C8 kann die Aktivierung intern über activeElementsCollection und completionCondition steuern oder einem Job Worker die Entscheidung überlassen.

Mechanismus	Bedeutung
activeElementsCollection	FEEL-Liste von Element-IDs, die beim Eintritt aktiviert werden.
completionCondition	Wird nach Abschluss innerer Elemente ausgewertet.
cancelRemainingInstances	Beendet restliche aktive Instanzen bei erfüllter Completion Condition, standardmäßig true.

Mechanismus	Bedeutung
Job-Worker-Modus	Worker wählt nächste Elemente und entscheidet über Abschluss.
Einschränkung	Bei interner Steuerung können Elemente derzeit nicht nachträglich dynamisch aktiviert werden, nur beim Eintritt.

EINSATZGRENZE: Ad-hoc eignet sich für flexible Wissensarbeit mit klaren verfügbaren Aktivitäten. Es ist kein Ersatz für unmodellerte Beliebigkeit; Aktivierungslogik, Abschlusskriterium und Datenkonflikte bleiben explizit zu gestalten.

7.5 Auswahlhilfe für Subprozessformen

Bedarf	Passender Baustein	Begründung
Lokaler Scope und gemeinsame Fehlergrenze	Embedded Subprocess	Kapselt Variablen und erlaubt Boundary Events ohne neue Prozessinstanz.
Wiederverwendung und eigenständiger Lifecycle	Call Activity	Startet eine referenzierte Prozessdefinition mit bewusstem Binding und Datenvertrag.
Reaktion auf Ereignis für einen gesamten Scope	Event Subprocess	Liegt innerhalb des Scopes und kann interrupting oder non-interrupting reagieren.
Flexible Reihenfolge optionaler Tätigkeiten	Ad-hoc Subprocess	Aktivitäten werden intern oder durch Worker ausgewählt; Completion bleibt ausdrücklich.

8 Was fehlt oder anders funktioniert

Nicht ausführbarer Baustein: fachliche Absicht erhalten, Mechanik ersetzen

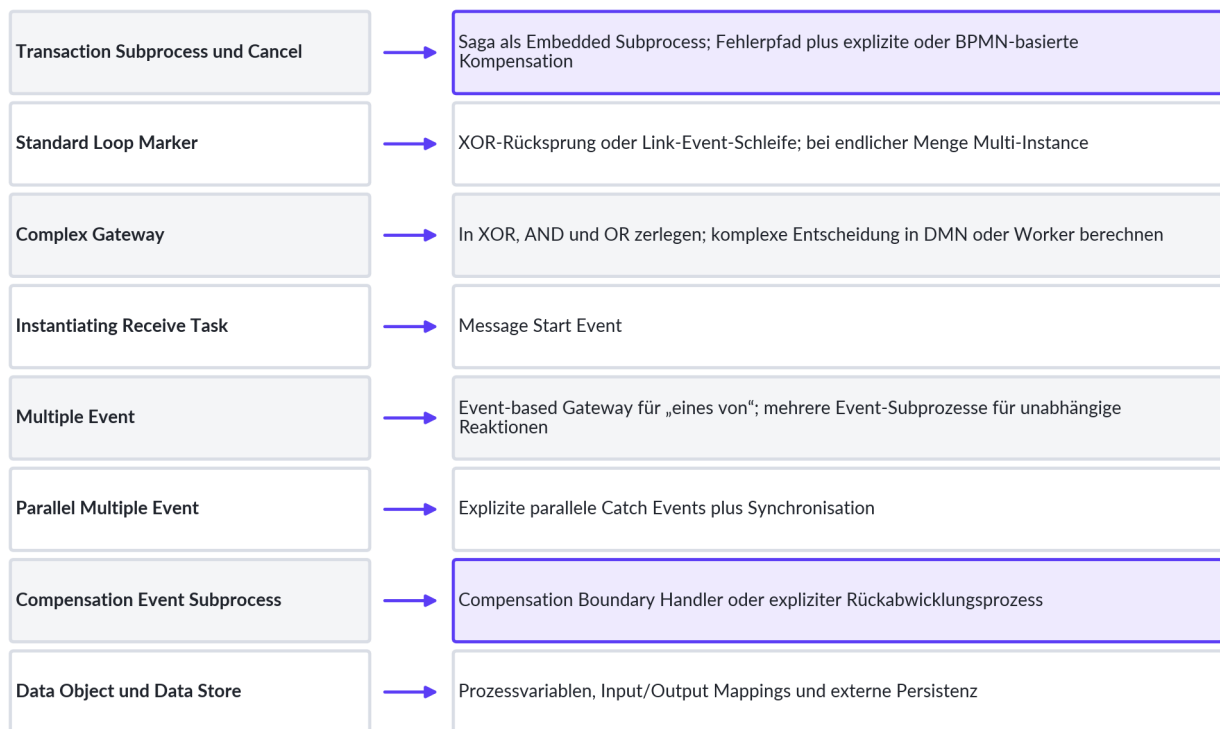
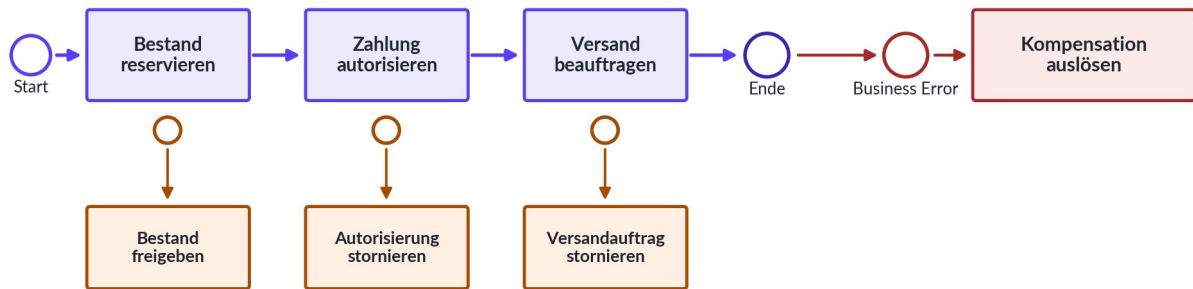


Abbildung 10: Ersatzmuster für nicht ausführbare Elemente

8.1 Transaction Subprocess und Cancel Events Quellen: C7-03, C8-01

Camunda 8.9 führt weder den Transaction Subprocess noch Cancel End oder Cancel Boundary Events aus. Das C7-Konstrukt muss nach seiner Absicht zerlegt werden. Meist lautet die Absicht: mehrere bereits erfolgreiche Schritte bei einem späteren fachlichen Scheitern rückgängig machen. In C8 wird daraus eine Saga in einem Embedded Subprocess mit fachlichem Error-Pfad und Kompensation.

C8-Muster: Saga mit modellierter Kompensation



C8 führt mehrere Handler ohne garantierte Reihenfolge aus.
Bei Reihenfolgeabhängigkeit gezielt per activityRef oder als expliziten Rückabwicklungsfluss orchestrieren.

Abbildung 11: Saga mit Kompensations-Handlern

C7-Absicht	C8-Modellierung
Cancel End löst Rückabwicklung aus	BPMN Error oder explizite Entscheidung führt zu Compensation Throw oder Rückabwicklungs-Subprozess.
Cancel Boundary verlässt Transaction Scope	Error Boundary/Event Subprocess am Embedded Subprocess; danach fachlicher Alternativpfad.
Automatische Rückwärtsreihenfolge	Reihenfolge in C8 explizit mit activityRef oder sequenziellem Rückbaufloss modellieren.
Technische Rollback-Erwartung	Jede externe Operation idempotent und kompensierbar gestalten; keine gemeinsame ACID-Transaktion annehmen.

BEGRIFFSGRENZE: Der BPMN Transaction Subprocess war auch in C7 kein universeller Distributed-Transaction-Manager. Er modellierte Cancel und Compensation. Die gemeinsam eingebettete ACID-Transaktion war ein separates Architekturmerkmal.

8.2 Compensation Event Subprocess

C7 kann unter Einschränkungen einen Compensation Start Event Subprocess als Handler eines Embedded Subprocess verwenden. C8.9 unterstützt diese Ereignisplatzierung nicht. Die fachliche Rückabwicklung wird entweder über Compensation Boundary Events an den relevanten Aktivitäten oder über einen expliziten Rückabwicklungsprozess modelliert.

8.3 Complex Gateway

Das Complex Gateway ist weder in C7.24 noch in C8.9 enginefähig. Bei einer komplexen Auswahl wird die Entscheidung vor dem Gateway in DMN, FEEL oder einem Worker berechnet und anschließend mit XOR, OR und AND sichtbar umgesetzt. Bei komplexer Synchronisation wird ein explizites Aggregationsmuster mit Zähler, Multi-Instance Completion Condition oder separatem Koordinations-Worker verwendet.

8.4 Multiple und Parallel Multiple Events

Fachliche Anforderung	C8-Muster
Der erste von mehreren Auslösern gewinnt	Event-based Gateway mit separaten Message-, Timer- oder Signal-Catch-Events.
Jeder Auslöser soll unabhängig eine Reaktion starten	Mehrere non-interrupting Event Subprocesses oder Boundary Events.
Alle unterschiedlichen Ereignisse müssen eintreffen	Parallele Pfade mit jeweils einem Catch Event und Parallel Join.
Unterschiedliche Startarten sind zulässig	Mehrere explizite Start Events, soweit die Ereignistypen unterstützt werden.

8.5 Instantiating Receive Task

Der instantiierende Receive Task ist in beiden Vergleichsversionen nicht ausführbar. Eine Nachricht, die eine neue Instanz erzeugt, wird als Message Start Event modelliert. Soll zunächst ein technischer Eingang und danach ein benannter Wartezustand dargestellt werden, können Start Event und regulärer Receive Task getrennt modelliert werden.

8.6 Data Object und Data Store Quellen: C8-01

Data Objects und Data Stores sind in C8 für die Modellierung sichtbar, haben aber keine Engine-Wirkung. Ein Data Store bedeutet nicht, dass Zeebe dort Daten persistiert. Operative Daten werden als Prozessvariablen transportiert oder in externen Systemen gespeichert; Worker und Connectoren lesen oder schreiben diese Systeme. Input/Output Mappings dokumentieren und begrenzen den Datenfluss.

8.7 asyncBefore, asyncAfter und exklusive Jobs

C8 besitzt keine direkten Pendanten zu den C7-Attributen `asyncBefore`, `asyncAfter` und `exclusive`. Ein Service Task und ein job-basierter Listener sind bereits persistente Wartepunkte. Benötigt das Modell eine weitere technische Grenze, wird eine explizite Aktivität oder ein Listener eingeführt. Konkurrenzschutz wird nicht über ein exklusives Job-Flag erwartet, sondern über Scope-Design, lokale Variablen, atomare externe Operationen und Idempotenz.

C7-Anwendungsfall	C8-Lösung
Transaktion vor teurem Aufruf committen	Aufruf als Service Task/Connector modellieren; Job ist die Grenze.
Nach einer Aktivität asynchron fortsetzen	Expliziten nachgelagerten Service Task oder End Listener verwenden, wenn fachlich/technisch nötig.
Parallele Jobs derselben Instanz serialisieren	Modellfluss sequenzialisieren oder externe Ressource idempotent/optimistisch sperren.
Listener-Klasse ausführen	Execution Listener mit Job Type und Worker.

8.8 Engine- und UI-Plugins Quellen: MIG-01

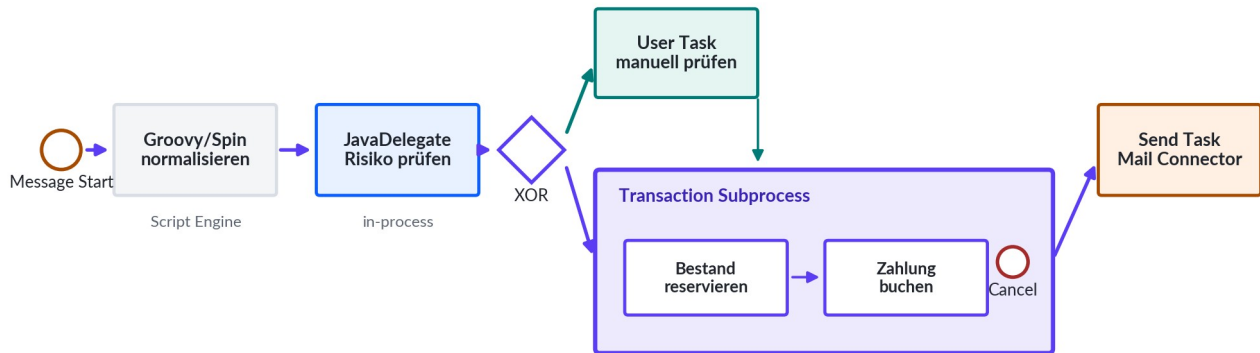
Process Engine Plugins sowie Cockpit- oder Tasklist-Plugins aus C7 können nicht als gleichartige Erweiterung nach C8 migriert werden. Je nach Zweck kommen Exporter, Interceptor, API-basierte Anwendungen, Worker oder explizite Modellschritte infrage. Exporter stehen nur in Self-Managed-Zeebe-Clustern zur Verfügung. Diese Änderung ist nicht primär ein BPMN-Thema, beeinflusst aber, welche „unsichtbare“ Logik künftig im Modell oder in klaren Integrationskomponenten erscheinen muss.

9 Durchgängiges Fallbeispiel

9.1 Ausgangslage

Ein Bestellprozess startet per Nachricht, normalisiert Daten mit Groovy und Spin, ruft eine Risiko-Bean auf, verzweigt per JUEL, fordert bei hohem Risiko eine manuelle Prüfung an und führt Reservierung plus Zahlung in einem Transaction Subprocess aus. Bei Abbruch werden Schritte kompensiert. Abschließend versendet ein Camunda-7-Connector eine E-Mail.

Fallbeispiel: typische Camunda-7-Lösung



Typische technische Kopplungen: JUEL, Delegate Expressions, asyncBefore/After, eingebettete Forms.

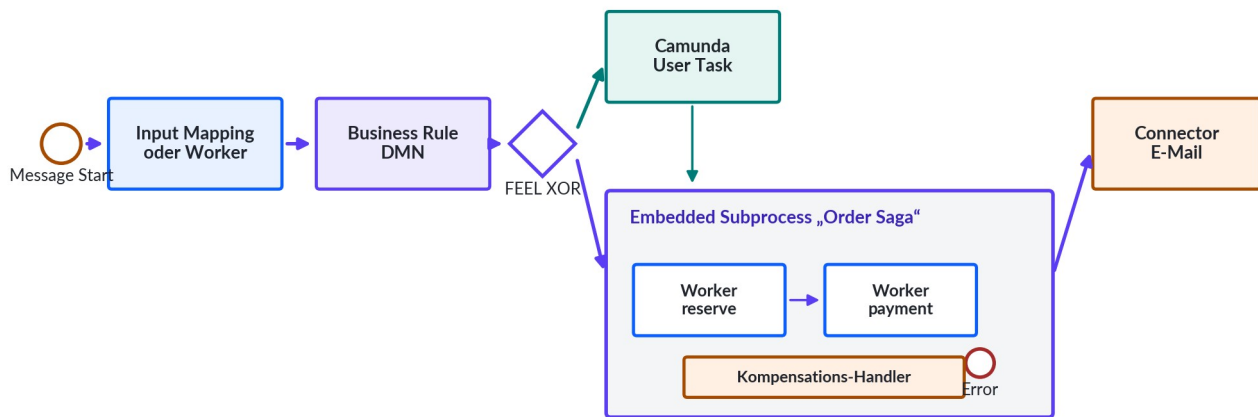
Abbildung 12: Vereinfachtes C7-Ausgangsmodell

9.2 Analyse des C7-Modells

Fundstelle	Klassifikation	C8-Konsequenz
Groovy/Spin Script Task	Anders konfigurieren	Kleine Transformation als FEEL; sonst Worker mit eigener JSON-Transformation.
JavaDelegate Risiko	Implementierung austauschen	DMN Business Rule Task oder Worker.
JUEL Gateway	Expression umschreiben	FEEL-Bedingung und Default Flow.
Embedded Form/User Task	Ressource neu binden	Camunda User Task plus Camunda Form oder externe Form.
Transaction Subprocess/Cancel	Nicht ausführbar in C8	Saga plus Error und Compensation.
asyncBefore Zahlung	Kein direktes Attribut	Service-Task-Job ist bereits Grenze; Retry/Idempotenz definieren.
C7 Mail Connector	Infrastruktur anders	C8 Connector Template oder Worker.

9.3 Zielmodell

Fallbeispiel: Camunda-8-Zielmodell



Technischer Fehler: Fail, Retries, Incident. Fachlicher Fehler: BPMN Error.
Datenvertrag über FEEL und Input/Output Mappings; Ressourcenbindung vorzugsweise „deployment“.

Abbildung 13: C8-Zielmodell

9.4 Schrittweise Transformation

1. Startvertrag festlegen: Message Name order.received; Correlation Key ist die Order-ID; Sender nutzt TTL und Idempotenz.
2. Datenvertrag normalisieren: Prozessvariable order als JSON-Kontext; nur erforderliche Felder per Input Mapping an Worker/DMN geben.
3. Risikoentscheidung als DMN mit deployment Binding modellieren; Result Variable riskResult.
4. Gateway-Bedingungen in FEEL formulieren und einen fachlich vollständigen Default Flow setzen.
5. User Task als Camunda User Task mit Form-Binding deployment, candidateGroups als FEEL-Liste und klaren Output Mappings konfigurieren.
6. Transaction Subprocess durch Embedded Subprocess Order Saga ersetzen. Erfolgreiche Seiteneffekte erhalten Compensation Boundary Handler.
7. Worker unterscheiden: fachliche Ablehnung als BPMN Error; technische Störung als Fail mit Retries; alle externen Befehle idempotent.
8. E-Mail als Connector-Template konfigurieren. Secrets bleiben außerhalb des BPMN; nur fachliche Eingaben werden gemappt.
9. Happy Path, fachlichen Fehler, technischen Retry, Incident Recovery, doppelte Nachricht und parallele Variablenzugriffe testen.

9.5 Beispielhafte FEEL-Konfiguration

C8-Ausdrücke

```

Gateway „manuelle Prüfung nötig“:
= riskResult.decision = "REVIEW" or order.total > 1000

Candidate groups:
= if order.region = "EU" then ["risk-eu"] else ["risk-global"]

Worker input context:
= { orderId: order.id, amount: order.total, currency: order.currency }

Output mapping:
payment = { status: result.status, authorizationId: result.id }
  
```

9.6 Testfälle als Modellierungsbeweis

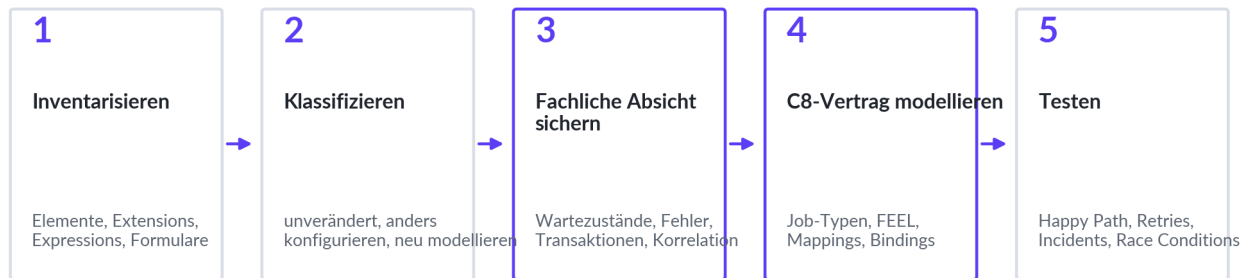
Testfall	Erwartete C8-Reaktion
Zahlung erfolgreich	Job Complete; Saga läuft weiter; Compensation Subscription bleibt bis Scope-Ende.
Karte fachlich abgelehnt	Worker wirft Error PAYMENT_DECLINED; Error-Pfad startet Rückabwicklung.
Payment API 503	Worker Fail; Retry. Nach Ausschöpfung Incident, kein fachlicher Error-Pfad.
Gleiche order.received Message zweimal	Idempotenz/Correlation verhindert doppelte aktive Bestellung oder Worker erkennt Duplikat.
Bestand reserviert, Zahlung abgelehnt	Bestand wird kompensiert; Reihenfolge ist explizit modelliert.
Manuelle Prüfung läuft über Frist	Timer Boundary/Event Subprocess übernimmt Eskalation oder Abbruch gemäß Modell.

9.7 Abnahmekriterien des Zielmodells

Dimension	Nachweis
Semantik	Für Happy Path, fachliche Ablehnung, Timeout und Abbruch existieren eindeutige BPMN-Pfade.
Resilienz	Retries sind endlich; Worker-Operationen sind idempotent; Incident-Recovery ist beschrieben.
Datenvertrag	Input/Output Mappings begrenzen den Kontext; parallele Pfade überschreiben keine gemeinsamen Werte.
Korrelation	Message Name, Correlation Key, TTL und Duplikatverhalten sind getestet.
Betrieb	Operate-Sicht, Fehlercodes und Verantwortlichkeiten für Incidents sind festgelegt.

10 Checklisten, Review und Übungen

Arbeitsablauf für die Modellüberarbeitung



Nicht XML konvertieren und fertig. Zuerst die Ausführungssemantik prüfen.

Abbildung 14: Arbeitsablauf für die Überarbeitung

10.1 C8-Readiness-Check pro BPMN-Modell

Nr.	Prüfpunkt	Befund / Maßnahme
1	Execution Platform und Version sind korrekt gesetzt.	
2	Kein nicht ausführbares Element verbleibt: Transaction, Cancel, Complex Gateway, Standard Loop, Multiple Event, Instantiating Receive Task, Compensation Event Subprocess.	
3	Jeder Service/Send/Script/Business Rule Task besitzt einen eindeutigen C8-Ausführungsvertrag.	
4	Alle JUEL-Ausdrücke sind als FEEL oder explizite Arbeit neu gestaltet.	
5	Messages haben stabile Namen, Korrelationsschlüssel und eine begründete TTL-Strategie.	
6	Fachliche Fehler, technische Failures und Incidents sind getrennt.	
7	Parallele Pfade und Multi-Instance schreiben keine konkurrierenden Parent-Variablen.	
8	Call Activities und DMN/Form-Ressourcen haben ein bewusstes Binding.	
9	User Tasks verwenden den Camunda-User-Task-Typ und vollständige Assignment-/Form-Properties.	
10	Kompensation ist auf Reihenfolge, Multi-Instance und Call-Activity-Grenzen geprüft.	
11	Worker-Operationen sind idempotent; doppelte Jobs/Nachrichten sind im Test berücksichtigt.	
12	Nur fachlich erforderliche Daten werden per Input/Output Mapping transportiert.	

10.2 Task-Review

Frage	Ja/Nein	Notiz
Ist der BPMN-Task-Typ fachlich korrekt, unabhängig von der Implementierung?		
Ist der Job Type stabil und capability-orientiert?		
Sind Retries, Timeout, Backoff und Idempotenz abgestimmt?		
Sind Input und Output minimal und lokal gescoped?		
Ist ein fachlicher Error Code mit Catch Event abgestimmt?		
Sind Secrets und technische Endpunkte außerhalb des Modells konfiguriert?		

10.3 Message-Review

Frage	Ja/Nein	Notiz
Beschreibt der Message Name ein stabiles fachliches Ereignis?		
Identifiziert der Correlation Key genau eine erwartete Instanz?		
Kann die Nachricht vor dem Abonnement eintreffen; ist TTL passend?		
Sind Duplikate und Wiederholung sicher?		
Werden nur benötigte Variablen mit der Nachricht gemergt?		
Ist Signal wirklich Broadcast oder wäre Message präziser?		

10.4 Fehler- und Kompensationsreview

Frage	Ja/Nein	Notiz
Ist die Abweichung fachlich erwartet oder technisch?		
Werfen Worker nur für modellierte Business Errors einen BPMN Error?		
Sind Retries bei technischen Fehlern endlich und betrieblich sinnvoll?		
Sind Compensation Handler idempotent?		
Ist die Reihenfolge der Rückabwicklung in C8 explizit, falls relevant?		
Ist Multi-Instance-Kompensation als einmaliger Handler berücksichtigt?		
Stoppt Kompensation an einer Call Activity; gibt es dort einen Handler?		

10.5 Typische Anti-Patterns

Anti-Pattern	Problem	Bessere C8-Gestaltung
Jedes C7-Rechteck unverändert importieren	Symbolgleichheit verdeckt falsche Extensions.	Jedes Element nach Semantik, Daten und Ausführungsvertrag klassifizieren.
Technische Fehler als BPMN Error	Retries und Incident-Betrieb werden umgangen.	Fail für technische Fehler; Error nur für fachliche Pfade.
Alle Variablen überall propagieren	Kopplung, Datenleck und Race Conditions.	Lokale Scopes und explizite Mappings.
latest für alle Ressourcen	Laufende Prozesse können ungetestete neue Ressourcen aufrufen.	deployment oder versionTag, sofern Reproduzierbarkeit zählt.
Manual Task als menschlicher Wartezustand	Engine läuft sofort weiter.	User Task oder Message/Timer-Wartezustand.
Kompensation als magisches Rollback	Externe Seiteneffekte sind nicht automatisch reversibel.	Explizite, idempotente Gegenoperationen und Reihenfolge.
Große Geschäftslogik in FEEL	Schwer testbar und unübersichtlich.	DMN oder Worker; FEEL für Datenlogik und Bedingungen.

10.6 Übungen

Übung 1: Fehlerklassifikation

Ein Worker erhält HTTP 503, eine Kreditkarte wird abgelehnt und ein FEEL-Ausdruck greift auf order.customer.id zu, obwohl customer fehlt. Ordnen Sie Fail, BPMN Error und Incident zu.

Arbeitsraum: _____

Übung 2: Transaction ersetzen

Ein C7 Transaction Subprocess bucht Hotel, Flug und Mietwagen; Cancel soll alles rückgängig machen. Entwerfen Sie ein C8-Saga-Modell mit definierter Reihenfolge.

Arbeitsraum: _____

Übung 3: Schleife

Ein Prüfschritt soll höchstens dreimal wiederholt werden, bis status = "OK". Modellieren Sie eine sichtbare C8-Schleife und begründen Sie, wann Multi-Instance ungeeignet ist.

Arbeitsraum: _____

Übung 4: Message oder Signal

Eine Preisänderung soll alle laufenden Angebotsprozesse informieren; eine Zahlungsbestätigung genau eine Bestellung. Wählen und begründen Sie die Ereignisse.

Arbeitsraum: _____

Übung 5: Parallelität

Zehn Multi-Instance-Worker schreiben jeweils result in den Parent Scope. Erklären Sie das Risiko und konfigurieren Sie outputElement/outputCollection.

Arbeitsraum: _____

10.7 Musterlösungen

1. Übung 1: HTTP 503 wird als Fail behandelt und führt über Retries gegebenenfalls zum Incident. Die fachliche Kartenablehnung ist ein BPMN Error mit modelliertem Catch Event. Die fehlende Variable verursacht bei der Expression-Auswertung einen Incident.
2. Übung 2: Embedded Subprocess mit drei Service Tasks. Jeder erfolgreiche Schritt erhält einen Compensation Handler. Der fachliche Fehlerpfad löst Kompensation gezielt in der benötigten Reihenfolge aus oder verwendet einen sequenziellen Rückabwicklungs-Subprozess.
3. Übung 3: Zähler attempt lokal führen; nach dem Prüfschritt XOR mit status = "OK", attempt < 3 und Default-Abbruch. Multi-Instance ist ungeeignet, weil die nächste Wiederholung vom Ergebnis der vorherigen abhängt.
4. Übung 4: Preisänderung ist ein Signal mit 1:N-Broadcast. Zahlungsbestätigung ist eine Message mit Name und Order-ID als Correlation Key.
5. Übung 5: result wird von parallelen Instanzen überschrieben. result lokal pro Instanz halten; outputElement = result und outputCollection = results verwenden.

10.8 Bewertungsraster für Übungsergebnisse

Kriterium	2 Punkte	1 Punkt	0 Punkte
Fachliche Semantik	Wartezustände, Fehler und Rückabwicklung sind eindeutig.	Grundidee stimmt, einzelne Pfade bleiben mehrdeutig.	Technische Details ersetzen die fachliche Semantik.
C8-Ausführungsvertrag	Job-Typen, Bindings, Mappings und Korrelation sind vollständig.	Wesentliche Properties vorhanden, Lücken im Daten- oder Ressourcenvertrag.	Elemente sind nur gezeichnet, nicht ausführbar konfiguriert.
Fehler und Betrieb	Business Error, Fail, Retry und Incident sind getrennt.	Trennung teilweise vorhanden.	Jede Abweichung wird gleich behandelt.
Testbarkeit	Happy Path, Fehler, Duplikate und Parallelität sind prüfbar.	Nur Hauptpfade sind testbar.	Keine überprüfbaren Erwartungen formuliert.

A Vollständige Schnellreferenz

A.1 C8.9-Ereignisplatzierungen Quellen: C8-01

Event	Start	Event Subprocess	Catch	Boundary int.	Boundary non-int.	Throw	End
None	Ausführbar	Nicht vorgesehen	Nicht vorgesehen	Nicht vorgesehen	Nicht vorgesehen	Ausführbar	Ausführbar
Message	Ausführbar	Ausführbar	Ausführbar	Ausführbar	Ausführbar	Ausführbar	Ausführbar
Timer	Ausführbar	Ausführbar	Ausführbar	Ausführbar	Ausführbar	Nicht vorgesehen	Nicht vorgesehen
Conditional	Ausführbar	Ausführbar	Ausführbar	Ausführbar	Ausführbar	Nicht vorgesehen	Nicht vorgesehen
Signal	Ausführbar	Ausführbar	Ausführbar	Ausführbar	Ausführbar	Ausführbar	Ausführbar
Error	Nicht vorgesehen	Ausführbar	Nicht vorgesehen	Ausführbar	Nicht vorgesehen	Nicht vorgesehen	Ausführbar
Escalation	Nicht vorgesehen	Ausführbar	Nicht vorgesehen	Ausführbar	Ausführbar	Ausführbar	Ausführbar
Compensation	Nicht vorgesehen	Nicht ausführbar	Nicht vorgesehen	Ausführbar	Nicht vorgesehen	Ausführbar	Ausführbar
Cancel	Nicht vorgesehen	Nicht vorgesehen	Nicht vorgesehen	Nicht ausführbar	Nicht vorgesehen	Nicht vorgesehen	Nicht ausführbar
Terminate	Nicht vorgesehen	Nicht vorgesehen	Nicht vorgesehen	Nicht vorgesehen	Nicht vorgesehen	Nicht vorgesehen	Ausführbar
Link	Nicht vorgesehen	Nicht vorgesehen	Ausführbar	Nicht vorgesehen	Nicht vorgesehen	Ausführbar	Nicht vorgesehen
Multiple	Nicht ausführbar	Nicht ausführbar	Nicht ausführbar	Nicht ausführbar	Nicht ausführbar	Nicht ausführbar	Nicht ausführbar
Parallel Multiple	Nicht ausführbar	Nicht ausführbar	Nicht ausführbar	Nicht ausführbar	Nicht ausführbar	Nicht ausführbar	Nicht ausführbar

A.2 Bausteinstatus kompakt

Familie	C8.9 ausführbar	Nicht ausführbar oder nur Modellierung
Tasks	Service, User, Receive, Send, Business Rule, Script, Manual, Undefined	Instantiating Receive Task
Gateways	Exclusive, Parallel, Inclusive, Event-based	Complex
Subprozesse	Embedded, Call Activity, Event Subprocess, Ad-hoc	Transaction
Marker	Parallel MI, Sequential MI, Compensation, Ad-hoc	Standard Loop
Events	None, Message, Timer, Conditional, Signal, Error, Escalation, Terminate, Link, Compensation	Cancel, Multiple, Parallel Multiple; Compensation Event Subprocess
Daten/Artefakte	Data Object, Data Store, Text Annotation, Group nur zur Modellierung	Keine Engine-Datenwirkung

B Property-Mapping C7 nach C8

C7-Property/Konzept	C8-Ziel	Prüfhinweis
camunda:class	zeebe:taskDefinition type	Worker registriert sich auf Typ; Klasse bleibt außerhalb BPMN.
camunda:delegateExpression	zeebe:taskDefinition type	Spring Bean wird Job Worker.
camunda:expression	FEEL oder Worker	Keine Bean-Aufrufe in FEEL.
camunda:type="external"	Service Task Job Type	External-Task-Prinzip ist Standard.
camunda:topic	Job Type	Benennung und Routing neu prüfen.
camunda:asyncBefore / asyncAfter	Kein direktes Pendant	Expliziter Job-/Listener-Schritt bei Bedarf.
camunda:exclusive	Kein direktes Pendant	Parallelität und Idempotenz im Modell/Worker lösen.
camunda:inputOutput	zeebe:ioMapping	FEEL Sources/Targets; Scope-Wirkung testen.
\${...} JUEL	= ... FEEL	Syntax und Semantik neu schreiben.
camunda:formKey / embedded form	zeebe:formDefinition	Camunda Form oder externe Referenz plus Binding.
candidateGroups String/JUEL	candidateGroups FEEL-Liste	IDs, Case-Sensitivity und Identity prüfen.
TaskListener class/expression	User Task Listener type	Job-basierter Lifecycle Hook.
ExecutionListener class/expression	Execution Listener type	Start/End Listener als blockierender Job.
calledElement	zeebe:calledElement processId	Binding und Variable Propagation konfigurieren.
decisionRef	zeebe:calledDecision decisionId	Binding und Result Variable konfigurieren.
Java-/Spin-Variable	JSON/Primärtyp	Transformation im Worker/Anwendungscode.

C FEEL-Kurzübersicht

Absicht	FEEL-Beispiel
Ausdruck kennzeichnen	= order.total > 1000
Boolesch kombinieren	= customer.vip and order.total > 1000
Kontextfeld lesen	= order.customer.id
Liste erzeugen	= ["risk-eu", "supervisors"]
Liste enthält Wert	= list contains(allowedStatuses, status)
Liste filtern	= items[item.price > 100]
Kontext erzeugen	= { id: order.id, amount: order.total }
Bedingter Wert	= if priority = "high" then 90 else 50
n-mal iterieren	= for i in 1..n return i
Datum/Dauer	= now() + duration("P2D")
Null prüfen	= value = null
Prozentuale MI-Completion	= numberOfCompletedInstances / numberOfInstances >= 0.8

HINWEIS: FEEL-Ausdrücke im Modeler beginnen je nach Property mit „=“. Statische Werte werden ohne „=“ eingetragen. Die Property-Dokumentation entscheidet, ob ein Ausdruck, ein statischer Wert oder beides zulässig ist.

D Glossar

Begriff	Bedeutung
Binding	Regel, welche Version einer referenzierten DMN-, Form- oder Prozessressource verwendet wird.
Business Error	Fachlich erwartete Ausnahme, die als BPMN Error in einen modellierten Pfad führt.
Connector	Vorkonfigurierte Integration, die im Modell meist über ein Element Template auf einem Service Task erscheint.
Correlation Key	Wert, mit dem eine Message einer erwartenden Prozessinstanz zugeordnet wird.
Element Template	Wiederverwendbare Modeler-Konfiguration, die Properties und UI eines Elements strukturiert.
Execution Listener	Start- oder End-Hook an BPMN-Elementen; in C8 als blockierender Job ausgeführt.
FEEL	Expression Language für Bedingungen, Mappings und Datenlogik in Camunda 8.
Incident	Betrieblicher Halt wegen nicht auflösbarer Ausführungssituation, etwa ausgeschöpften Retries oder Expression-Fehler.
Job Type	Routing-Schlüssel, über den Worker Jobs eines Service Tasks abonnieren.
Local Variable	Variable, deren Sichtbarkeit auf einen Scope oder eine Elementinstanz begrenzt ist.
Message TTL	Zeit, in der eine noch nicht korrelierbare Message gepuffert werden kann.
Process Application	Zusammengehörige Prozess-, Entscheidungs-, Form- und Integrationsartefakte als Lösungseinheit.
Saga	Folge lokaler Transaktionen mit Gegenoperationen zur fachlichen Konsistenz ohne verteilte ACID-Transaktion.
Worker	Externe Anwendungskomponente, die Jobs aktiviert, Arbeit ausführt und Complete, Fail oder BPMN Error meldet.
Zeebe	Workflow Engine und Orchestrierungskern von Camunda 8.

E Quellen und Versionshinweise

Die fachlichen Aussagen dieses Handbuchs wurden gegen die offiziellen Camunda-Dokumentationen für Camunda 8.9 und Camunda 7.24 geprüft. Quellen-IDs im Text verweisen auf die folgende Liste. Produktfunktionen ändern sich zwischen Minor-Versionen; bei einem Upgrade ist insbesondere die aktuelle BPMN-Coverage erneut zu prüfen.

C8-01 [Camunda 8.9: BPMN coverage](#)

C7-01 [Camunda 7.24: BPMN 2.0 Implementation Reference](#)

MIG-01 [Conceptual differences between Camunda 7 and Camunda 8](#)

MIG-02 [Migration tools](#)

C8-02 [Service tasks](#)

C8-03 [User tasks](#)

- [C8-04 Receive tasks](#)
- [C8-05 Business rule tasks](#)
- [C8-06 Script tasks](#)
- [C8-07 Send tasks](#)
- [C8-08 Manual tasks](#)
- [C8-09 Undefined tasks](#)
- [C8-10 Data flow](#)
- [C8-11 Multi-instance](#)
- [C8-12 Message events](#)
- [C8-13 Signal events](#)
- [C8-14 Timer events](#)
- [C8-15 Conditional events](#)
- [C8-16 Error events](#)
- [C8-17 Escalation events](#)
- [C8-18 Terminate events](#)
- [C8-19 Link events](#)
- [C8-20 Compensation events](#)
- [C8-21 Call activities](#)
- [C8-22 Event subprocesses](#)
- [C8-23 Ad-hoc subprocesses](#)
- [C8-24 Execution listeners](#)
- [C7-02 Camunda 7.24: Cancel and compensation events](#)
- [C7-03 Camunda 7.24: Transaction subprocess](#)

E.1 Abgrenzungen

- Das Handbuch beschreibt Modellierung und Ausführungssemantik. Installation, Clusterbetrieb, Security, API-Details und vollständige Migrationswerkzeuge werden nur dort behandelt, wo sie die Modellierung direkt beeinflussen.
- „Nicht ausführbar“ bezieht sich auf Camunda 8.9 bzw. Camunda 7.24 und nicht auf die vollständige BPMN-2.0-Spezifikation.
- Für konkrete Produktionsmodelle sind zusätzlich die im Unternehmen verwendete Camunda-8-Deploymentform, Identity-/Tasklist-Version, Connector-Laufzeit und Clientbibliotheken zu berücksichtigen.

ABSCHLUSS: Der zentrale Umstiegsschritt ist nicht das Austauschen von XML-Namensräumen. Er besteht darin, aus impliziter In-Process-Ausführung einen expliziten, verteilten und testbaren Orchestrierungsvertrag zu machen.